Desarrollo y evaluación de un Acelerador Criptográfico para el Algoritmo SHA-2 en Hardware Reconfigurable

1st Benjamín Oyarzún Valdebenito
Departamento de Ingeniería Eléctrica
Universidad de Santiago de Chile
Santiago, Chile
benjamin.oyarzun.v@usach.cl

2nd Nicolás Ruminot Ahumada
Departamento de Ingeniería Eléctrica
Universidad de Santiago de Chile
Santiago, Chile
nicolas.ruminot@usach.cl

3rd Christian Fernández-Campusano
Departamento de Ingeniería Eléctrica
Universidad de Santiago de Chile
Santiago, Chile
christian.fernandez@usach.cl

***Resumen*—El algoritmo criptográfico SHA-256 es fundamental en la seguridad de redes y en sistemas de consenso distribuido, donde la constante validación de bloques impone una elevada demanda computacional y energética. Este artículo presenta un análisis comparativo y experimental de dicho algoritmo sobre tres arquitecturas heterogéneas: un procesador multinúcleo Intel Core i7 con extensiones vectoriales SHA-NI, un acelerador gráfico masivamente paralelo NVIDIA GeForce RTX 2050 y un coprocesador dedicado desarrollado en Verilog RTL sobre una tarjeta FPGA AMD Spartan-7. A diferencia de estudios previos enfocados en minería sintética de un único bloque, la evaluación emplea una carga de trabajo homogénea de un gigabyte dividida en millones de mensajes independientes de sesenta y cuatro bytes, replicando entornos de red y árboles de Merkle. Bajo un protocolo de telemetría energética riguroso en bucles sostenidos, se cuantifica el impacto de la transferencia sobre el bus de interconexión, la memoria fija y la planificación dinámica libre de cerrojos. Los resultados empíricos revelan que la ventaja masiva de la GPU en cómputo puro queda limitada por el bus periférico en mensajes cortos, mientras que la CPU heterogénea alcanza la mayor eficiencia energética del sistema completo. La caracterización final en FPGA permanece pendiente (TODO).**

***Index Terms*—aceleradores, funciones hash, SHA-2, cómputo heterogéneo, FPGA.**

I. Introducción

Los Algoritmos de Hash Seguro (SHA), estandarizados por el NIST [1], [2], sustentan la integridad y autenticación en la infraestructura digital moderna. Entre ellos, SHA-256 es el estándar más extendido, garantizando desde protocolos TLS y verificación de árboles de Merkle hasta esquemas de consenso en redes blockchain [3], [4]. Esta omnipresencia conlleva un costo energético global masivo: solo la validación de transacciones y minería computacional demandó más de 119 millones de MWh en 2023 [5], subrayando la urgencia de maximizar tanto el rendimiento como la eficiencia energética por byte procesado.

Para procesar grandes volúmenes de hashing existen tres vías arquitectónicas principales: instrucciones vectoriales dedicadas en CPU (como Intel SHA-NI), aceleración masivamente paralela SIMT en GPU y circuitos integrados dedicados en FPGA/ASIC [6]. Sin embargo, la literatura existente adolece de un vacío crítico: la mayoría de los estudios evalúan aceleradores de forma aislada o se concentran en cargas de minería sintética de un solo mensaje largo sin considerar transferencias masivas de datos ni flujos de red continuos. Además, no existe una comparativa directa y sistemática de tres vías bajo un protocolo de telemetría energética riguroso y unificado que compute el consumo total del sistema incluyendo el procesador anfitrión.

Para abordar este vacío, este artículo presenta las siguientes contribuciones:

- **Evaluación heterogénea de tres vías:** Comparación directa de SHA-256 sobre CPU de consumo (Intel i7-1255U con SHA-NI), GPU móvil (NVIDIA RTX 2050) y FPGA reconfigurable (AMD Spartan-7 en RTL).
- **Carga de trabajo unificada:** Modelo experimental homogéneo enfocado en flujos masivos de mensajes cortos independientes ($C = 64$ B sobre 1 GiB de payload útil), representativo de validación de paquetes y bloques criptográficos.
- **Metodología energética integral:** Protocolo de telemetría unificado en bucles sostenidos (≥ 15 s) que distingue la potencia neta de solo dispositivo frente al sistema completo (dispositivo + host).
- **Análisis de cuellos de botella de interconexión:** Caracterización empírica del impacto del bus PCIe, técnicas de staging de memoria y planificación dinámica multihilo para arquitecturas heterogéneas.

II. Trabajo Relacionado

La literatura en aceleración de SHA-256 se estructura principalmente en tres ámbitos: implementaciones dedicadas en hardware reconfigurable, paralelismo masivo en coprocesadores gráficos y arquitecturas multinúcleo asistidas por extensiones vectoriales.

1) Aceleradores en FPGA y ASIC: Las técnicas arquitectónicas tradicionales para maximizar el throughput abarcan el desenrollado de bucles, la segmentación profunda (pipelining) y el reordenamiento espacial de etapas [7]–[9]. Diseños recientes han llevado estos esquemas a límites extremos: Li

et al. [10] alcanzaron 219.65 Gbps mediante un pipeline completo de 64 niveles en Virtex-7, mientras que Nannipieri et al. [11] demostraron en ASIC TSMC de 7 nm tasas superiores a 33 Gbps con disipaciones de solo 13.4 mW. En el extremo opuesto, orientado a nodos embebidos y plataformas de bajo consumo, se prioriza la minimización de área y potencia dinámica mediante plegado de bucles y conversión de reloj con compuertas [6], [12]–[14]. Complementariamente, diversas arquitecturas multimodo y plataformas de co-diseño HW/SW han explorado compromisos entre flexibilidad algorítmica y sobrecarga de multiplexación [15], [16]. Asimismo, cabe destacar que algunas comparaciones FPGA vs. CPU de la literatura contrastan sus diseños frente a CPUs sin extensiones SHA-NI (por ejemplo, Pham et al. [15] evalúan frente a un Intel Core i9-10940X carente de SHA-NI), mientras que este trabajo establece como línea base rigurosa una CPU con soporte nativo de SHA-NI.

2) Aceleración en GPU y extensiones vectoriales en CPU: La arquitectura masivamente paralela SIMT de las GPUs ha sido ampliamente explotada en esquemas de minería de bloques y hashes criptográficos concurrentes; Mehammed et al. [17], [18] caracterizaron la distribución de bloques de hilos, ocupación de registros y jerarquía de memoria en CUDA, reportando aceleraciones significativas frente a implementaciones secuenciales en CPU. Por su parte, en arquitecturas de CPU de propósito general, Gueron et al. [19] formalizaron el procesamiento multi-buffer vectorizado para amortizar latencias en el cálculo concurrente de mensajes independientes, principio que cristalizó en microarquitecturas comerciales mediante instrucciones dedicadas por hardware como Intel SHA Extensions (SHA-NI) [20].

3) Vacío en la literatura: Si bien diversas investigaciones han realizado comparativas entre subconjuntos de plataformas (tales como esquemas híbridos CPU-FPGA [21], dobles hashes para minería en CPU/GPU/FPGA [22], coprocesadores dedicados frente a procesadores comerciales [23] o análisis de tiempos de transferencia y cómputo [24]), la literatura carece de una comparativa sistemática de tres vías que enfrente simultáneamente CPU con extensiones SHA-NI, GPU con optimización SIMT y FPGA RTL dedicada bajo una carga homogénea de mensajes cortos independientes y empleando un protocolo de telemetría energética unificado de sistema completo.

III. Metodología

Para evaluar exhaustivamente el rendimiento y la eficiencia energética de SHA-256 en CPU, GPU y FPGA, se establece un marco experimental heterogéneo enfocado en el procesamiento masivo de mensajes cortos independientes (representativos de la verificación en árboles de Merkle, validación de paquetes y transacciones blockchain).

III-A. Workload Unificado y Métricas de Rendimiento

El conjunto de datos consiste en un archivo binario de 1 GiB (1 073 741 824 bytes, 2^{30} B) generado con un PRNG de semilla fija. Para modelar mensajes independientes, el archivo se particiona en:

$$N = \left\lfloor \frac{2^{30}}{C} \right\rfloor \tag{1}$$

fragmentos contiguos de C bytes. Siguiendo la especificación FIPS 180-4 [1], el procesamiento de cada mensaje requiere agregar un byte delimitador `0x80`, relleno de ceros y la longitud del mensaje en 64 bits (total: $C + 9$ bytes mínimos). El número de bloques de compresión de 64 bytes procesados por cada fragmento es:

$$B = \left\lceil \frac{C + 9}{64} \right\rceil \tag{2}$$

La salida de cada corrida consiste en el arreglo contiguo de los N digests resultantes ($N \times 32$ bytes en memoria). El escenario principal del estudio es $C = 64$ bytes ($N = 16\,777\,216$ fragmentos, $B = 2$ bloques por mensaje y salida de 512 MiB). Adicionalmente, se realiza un barrido paramétrico en CPU y GPU con $C \in \{64\text{ B},\ 1\text{ KB},\ 4\text{ KB},\ 64\text{ KB},\ 1\text{ MB}\}$.

Las métricas normalizadas de evaluación son:

- **Throughput útil [GB/s]:** Tasa de procesamiento del payload real ($N \cdot C$), excluyendo relleno y cabecera de longitud:

$$\text{Throughput} = \frac{N \cdot C}{t \cdot 10^9} \quad [\text{GB/s}] \tag{3}$$

  donde t es el tiempo de ejecución en segundos. Complementariamente, las tasas de procesamiento aritmético se obtienen como hashes/s $= N/t$ y bloques/s $= (N \cdot B)/t$.
- **Eficiencia energética [J/GB]:** Energía neta consumida por gigabyte útil procesado:

$$\text{J/GB} = \frac{\Delta E}{\text{iteraciones} \cdot (N \cdot C/10^9)} \tag{4}$$

  donde ΔE representa la energía neta ($\Delta E_{\text{neta}} = \Delta E_{\text{bruta}} - P_{\text{reposo}} \cdot t$).
- **Factor de Aceleración (Speedup):** Se establece una única definición de referencia en todo el documento:

$$\text{Speedup} = \frac{\text{Throughput}_{\text{plataforma}}}{\text{Throughput}_{\text{SHA-NI 4T}}} \tag{5}$$

  anclada en la configuración CPU SHA-NI con 4 hilos extremo a extremo (E2E, *end-to-end*) sobre el escenario principal ($C = 64$ B, throughput medido de 3.55 GB/s).

La métrica extremo a extremo (E2E) abarca el flujo completo: en CPU comprende la lectura secuencial de fragmentos precargados en memoria RAM y la escritura contigua de digests en RAM (RAM→RAM); en GPU incluye la transferencia H2D del payload vía PCIe, la ejecución del kernel y la transferencia D2H de los digests hacia el anfitrión. En el código fuente, la modalidad pinned preasigna y fija los buffers antes del bucle de medición (sin copia intermedia en el tiempo medido), mientras que la modalidad pageable incluye explícitamente la llamada síncrona `memcpy` de 1 GiB desde la memoria virtual al buffer pinned dentro del intervalo temporal y energético medido.

III-B. Protocolo de Medición y Telemetría

Cada configuración se evalúa tras 3 s de calentamiento mediante bucles continuos de $\geq$ 15 s por corrida a lo largo de 5 repeticiones consecutivas (reportando media $\pm$ desviación estándar). En CPU, la potencia del paquete se adquiere mediante el contador Intel RAPL en el dominio de encapsulado completo (`PKG`, *package*) sobre el límite operativo PL1=20.0 W (Constraint 0 MMIO, sin eventos de estrangulamiento térmico), deduciendo el consumo basal en reposo (5.15 ± 0.06 W) para obtener la energía neta. En GPU, se utiliza el contador acumulativo de hardware `nvmlDeviceGetTotalEnergyConsumption` (descontando un reposo de 4.44 ± 0.29 W) y se registra simultáneamente el consumo RAPL PKG del host, permitiendo cuantificar la energía neta tanto del dispositivo aislado como del sistema completo GPU+host (excluyendo la DRAM del host). El hilo anfitrión de control de la GPU no cuenta con afinidad fijada por código ni `taskset`, ejecutándose libremente bajo la asignación del planificador del sistema operativo a través de las 12 CPUs lógicas del host (0–11). La correctitud algorítmica se valida mediante los 4 vectores estándar del NIST y la comparación de todos los digests contra `hashlib.sha256` de Python.

III-C. Implementaciones de Línea Base (CPU)

La ejecución en CPU se evaluó en un Intel Core i7-1255U (Alder Lake, 10 nm, 2 P-cores + 8 E-cores, 12 hilos lógicos, TDP 15 W) con 16 GB de memoria DDR4 de doble canal y el governor en modo `powersave`. Los hilos de cómputo se afinan a los núcleos de alta eficiencia (E-cores, CPUs 4–11) mediante `taskset`, evaluándose además una configuración concurrente sobre las 12 CPUs lógicas del procesador (0–11, P+E). Se compilaron dos variantes usando GCC 15.2 (`-O3`):

- **Escalar (x86-64):** Instrucciones de propósito general sin aceleración criptográfica.
- **SHA-NI:** Instrucciones nativas Intel SHA Extensions (`-msha, -msse4.1`), ejecutando las rondas de compresión directamente sobre registros SSE de 128 bits.

El escalado se evaluó con 1, 4, 8 hilos, 12 hilos sobre los 8 E-cores (sobresuscripción) y 12 CPUs (P+E). Para mitigar el desbalance de cómputo por la asimetría microarquitectónica entre núcleos P y E, se implementó una cola de trabajo compartida basada en un contador atómico libre de cerrojos con tareas de 64K fragmentos (4 MiB por bloque de trabajo).

III-D. Ejecución Masivamente Paralela (GPU)

La implementación GPU se desplegó en una NVIDIA GeForce RTX 2050 móvil (GA107M, Ampere, `sm_86`, 16 SMs, 2048 núcleos CUDA, 4 GB GDDR6 a 96 GB/s, TDP 30 W), compilada con CUDA 12.4 (`nvcc -O3 -arch=sm_86`) sobre el controlador NVIDIA 595.99.02. El kernel se lanza con bloques de 256 hilos (`THREADS_PER_BLOCK = 256`); en modo kernel-only el grid está dimensionado a 384 bloques, mientras que en modo E2E el procesamiento se particiona en lotes de 64 MiB (67 108 864 B) por stream con grids dinámicos de hasta 4 096 bloques en $C = 64$ B.

Como línea base, el kernel v1 asigna un hilo CUDA por fragmento y almacena la expansión de mensajes en un arreglo `w[64]`, limitando la ocupación a 2 bloques por SM (33.3 % de los 1 536 hilos teóricos) debido al elevado consumo de registros ($\approx$100 registros/hilo). Para superar esta limitación, la implementación v2 incorpora cuatro optimizaciones críticas de kernel y transferencia:

1. **Ventana circular `w[16]`:** Reduce los registros vivos a $\leq$48/hilo (`--maxrregcount=48`), elevando los bloques residentes por SM de 2 a 5 (ocupación teórica de 83.3 %).
2. **Memoria host pinned:** Uso de `cudaMallocHost` para transferencias DMA asíncronas directas evitando buffers intermedios del driver.
3. **Pipeline triple asíncrono:** Uso de 3 streams CUDA concurrentes para superponer la transferencia H2D de lotes de 64 MiB, la ejecución del kernel y la transferencia D2H de digests.
4. **Bucle *grid-stride*:** Desacoplamiento del dimensionamiento de la malla respecto al total de fragmentos N, asegurando una distribución balanceada de bloques en los 16 SMs.

III-E. Amenazas a la Validez y Limitaciones

Las conclusiones experimentales de este trabajo deben interpretarse considerando las siguientes limitaciones metodológicas: (1) las evaluaciones se concentran en una única plataforma móvil de consumo (laptop con CPU Intel i7-1255U bajo límite PL1=20 W, GPU NVIDIA RTX 2050 móvil de 30 W y enlace PCIe Gen 4 $\times$4); (2) existe una asimetría en los perímetros de telemetría, ya que NVML engloba el consumo de la memoria GDDR6 integrada en la GPU, mientras que Intel RAPL PKG excluye el consumo de la memoria DRAM del host; (3) la verificación cruzada de potencia instantánea de NVML recurre al mismo sensor analógico que el contador acumulativo de energía del dispositivo; (4) la ventana de relajación del límite PL1 impuesta por la plataforma es de 39.98 s frente a ventanas de corrida de 15 s; (5) el análisis microarquitectónico de GPU se basa en métricas de compilación de `ptxas` y telemetría de eventos temporales sin perfilado granular de contadores hardware con NVIDIA Nsight Compute; (6) el hilo anfitrión de la GPU no tiene afinidad fijada y puede migrar entre núcleos P y E, lo que puede alterar la energía del host; (7) en modo pinned se asume que los datos se generan directamente en memoria fija (sin copia de staging dentro del intervalo medido); y (8) el governor `powersave` deja las frecuencias de CPU sin fijar, y no se registraron relojes ni razones de *throttling* de la GPU durante las corridas.

III-F. Diseño de Microarquitectura RTL

III-G. Optimizaciones de Ruta de Datos y Ruta de Control

IV. Resultados

A continuación se presentan y discuten los resultados experimentales obtenidos en las plataformas CPU y GPU bajo

el protocolo de telemetría descrito.

IV-A. Resultados de CPU

La Tabla I resume el desempeño de las configuraciones evaluadas en el procesador Intel Core i7-1255U bajo planificación dinámica. Durante las corridas, los E-cores operaron a ≈3.48 GHz (1T), ≈3.50 GHz (4T) y se estabilizaron en ≈2.77 GHz en 8T; en la configuración sobre las 12 CPUs (0–11, P+E), los P-cores promediaron 2 838 MHz y los E-cores 2 200 MHz. En todas las ejecuciones multinúcleo la potencia del paquete se estabilizó en ≈20 W gobernada estrictamente por el límite MMIO PL1=20.0 W (Constraint 0), registrando temperaturas moderadas (65–72 °C) y cero eventos de estrangulamiento térmico (`throttle_count = 0`).

Tabla I
RESULTADOS DE CPU SHA-256 (I7-1255U, $C = 64$ B, $N = 16\,777\,216$ FRAGMENTOS, $B = 2$ BLOQUES)

Configuración	Throughput (GB/s)	Mhash/s	J/GB neto	Speedup[a]
Escalar 1T	0.11 ± 0.00	1.71 ± 0.00	45.58 ± 0.05	0.03×
SHA-NI 1T	0.89 ± 0.00	13.90 ± 0.01	5.28 ± 0.02	0.25×
SHA-NI 4T (ref.)	3.55 ± 0.00	55.45 ± 0.01	4.13 ± 0.05	1.00×
SHA-NI 8T	5.60 ± 0.02	87.45 ± 0.35	2.67 ± 0.05	1.58×
SHA-NI 12 CPUs (P+E)	5.84 ± 0.02	91.25 ± 0.24	2.53 ± 0.02	1.65×

[a]Speedup vs. CPU SHA-NI 4T E2E. J/GB neto con $P_{\text{reposo}} = 5.15$ W (RAPL PKG).

La implementación escalar 1T establece la línea base sin aceleración con 0.11 GB/s y 45.58 J/GB netos. La variante SHA-NI 1T eleva el throughput a 0.89 GB/s reduciendo la energía neta a 5.28 J/GB. El escalado multihilo sobre E-cores alcanza 3.55 GB/s en 4T (4.13 J/GB netos) y 5.60 GB/s en 8T (2.67 J/GB netos), donde el rendimiento se estabiliza al saturar plenamente los 8 núcleos eficientes bajo el techo PL1. Asimismo, 12 hilos sobre los 8 E-cores (sobresuscripción) rinden 5.62 GB/s, equivalente a 8T (5.60 GB/s), lo que confirma que los 8 E-cores ya están saturados bajo el límite PL1.

Bajo la configuración heterogénea de 12 CPUs (P+E), la adopción de una cola de trabajo dinámica libre de cerrojos redujo el desbalance entre núcleos Golden Cove y Gracemont de 146.9 ms a 6.6 ms, elevando el throughput de 4.51 GB/s (particionamiento estático) a 5.84 GB/s (planificación dinámica) y posicionándola como la configuración de CPU más eficiente tanto en tasa útil como en consumo energético (2.53 J/GB netos).

El barrido paramétrico en función de C para SHA-NI 4T (representado en la curva de referencia de la Figura 1) muestra cómo el throughput asciende de 3.55 GB/s en $C = 64$ B ($B = 2$) hasta estabilizarse en 8.99 GB/s en $C = 1$ MB ($B = 16\,385$), mientras el consumo energético neto desciende a 1.58 J/GB. Esta ganancia responde a la dilución del bloque de relleno (que pasa de representar el 50 % del trabajo a $< 0.1\,\%$) y a la amortización de una sobrecarga residual de aproximadamente 21.2 % en mensajes mínimos, probablemente atribuible a la inicialización reiterada de las variables de estado ($H^{(0)}$) y a escrituras dispersas de digests en memoria.

IV-B. Resultados de GPU

La comparación entre los kernels v1 (base) y v2 (alta utilización) a lo largo del barrido de C se resume en la Tabla II y se ilustra en la Figura 1 (staging pinned). El kernel v2 domina ampliamente en concurrencia masiva: en $C = 64$ B logra 15.62 GB/s kernel-only (mejora de 3.8× sobre los 4.11 GB/s de v1), alcanzando un pico de 22.29 GB/s en $C = 64$ KB (7.3× vs. v1). Asimismo, el kernel v2 rinde 8.24 GB/s en $C = 4$ KB frente a 21.52 GB/s (1 KB) y 22.29 GB/s (64 KB), exhibiendo un comportamiento no monótono; como hipótesis (no verificada al no disponer de perfilado con Nsight Compute), esta degradación se atribuye a accesos no coalescentes, donde la separación entre hilos de C bytes y strides potencia de dos podrían causar conflictos en la caché L2 o en los canales de DRAM. En $C = 1$ MB, el throughput converge a ≈3.34–3.37 GB/s en régimen kernel-only y a 2.45 GB/s en modo E2E; se plantea como hipótesis que este comportamiento responde a una severa falta de paralelismo más que a la latencia de memoria: en modo kernel-only se ejecuta un único lanzamiento con solo $N = 1\,024$ hilos útiles (un hilo por fragmento sobre 1 GiB), ocupando apenas el 4.2 % de la capacidad del chip (16 SM × 1 536 = 24 576 hilos residentes); por su parte, en modo E2E, al particionar el trabajo en lotes de 64 MiB por stream, cada lanzamiento procesa únicamente 64 fragmentos (64 hilos), alcanzando con 3 streams concurrentes a lo sumo ≈192 hilos en vuelo ($< 0.8\,\%$ de ocupación). Cabe señalar además que asignar un hilo por mensaje no resulta representativo para procesar mensajes largos, donde el paralelismo intra-mensaje ofrecería una mayor saturación de la arquitectura.

Tabla II
BARRIDO DE TAMAÑO DE FRAGMENTO C EN CPU SHA-NI 4T Y GPU (KERNELS V1 Y V2)

C (B)	N (frag.)	B (bloques)	CPU SHA-NI 4T (GB/s)	GPU v1 kernel (GB/s)	GPU v2 kernel (GB/s)	GPU v2 E2E (GB/s)	Speedup E2E[a]
64	16 777 216	2	3.55 ± 0.00	4.11 ± 0.00	15.62 ± 0.15	4.98 ± 0.02	1.40×
1 024	1 048 576	17	8.21 ± 0.01	5.63 ± 0.02	21.52 ± 0.21	5.82 ± 0.00	0.71×
4 096	262 144	65	8.72 ± 0.12	3.76 ± 0.02	8.24 ± 0.04	5.87 ± 0.01	0.67×
65 536	16 384	1025	8.97 ± 0.01	3.06 ± 0.01	22.29 ± 0.10	5.36 ± 0.01	0.60×
1 048 576	1 024	16385	8.99 ± 0.00	3.34 ± 0.00	3.37 ± 0.00	2.45 ± 0.00	0.27×

[a]Speedup E2E = Throughput GPU v2 E2E pinned / Throughput CPU SHA-NI 4T.

La Tabla III cuantifica el impacto del staging de memoria en $C = 64$ B. Mientras que el throughput kernel-only es idéntico entre pinned y pageable (15.62 GB/s), el throughput E2E se degrada un 22.5 % en pageable (de 4.98 a 3.86 GB/s) debido a la copia intermedia síncrona en el driver, incrementando el consumo total del host de 1.99 a 2.57 J/GB netos (+29.1 %) y elevando el consumo del sistema completo de 5.48 J/GB a 6.34 J/GB netos.

Tabla III
IMPACTO DEL STAGING EN GPU V2 ($C = 64$ B)

Staging	Kernel-only (GB/s)	E2E (GB/s)	Ratio K/E2E	J/GB GPU	J/GB Host	J/GB Total
Pinned	15.62 ± 0.15	4.98 ± 0.02	3.13×	3.49 ± 0.04	1.99 ± 0.04	5.48 ± 0.07
Pageable	15.62 ± 0.15	3.86 ± 0.02	4.05×	3.77 ± 0.03	2.57 ± 0.06	6.34 ± 0.09

J/GB netos con P_{reposo} de 4.44 W (GPU) y 5.15 W (Host RAPL PKG, excluye DRAM).

El enlace opera en modo PCIe Gen 4 ×4, cuyo ancho de banda teórico simplex es de 7.88 GB/s (con-

siderando ya la codificación 128b/130b), reduciéndose a ≈6.4 GB/s útiles tras descontar la sobrecarga de tramas TLP y DLLP del protocolo. Con dos motores DMA independientes (`asyncEngineCount = 2`), la GPU soporta transferencias H2D y D2H simultáneas. Para $C = 64$ B con staging pinned, la GPU v2 alcanza 15.62 GB/s kernel-only frente a 4.98 GB/s E2E (ratio K/E2E de 3.13×), penalizada por la transferencia simultánea de 1 073.7 MB de datos y 536.9 MB de digests por iteración, donde se plantea la hipótesis de que la competencia bidireccional por los recursos del controlador de bus limita el rendimiento efectivo. En fragmentos medianos ($C = 1$–4 KB), la reducción del volumen de digests permite al throughput E2E alcanzar 5.87 GB/s (≈92 % del límite útil del bus).

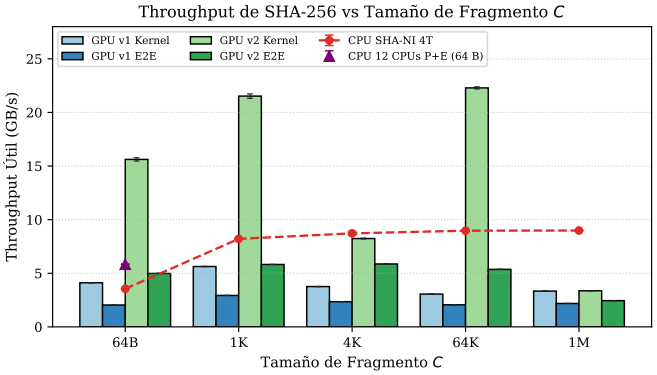


Figura 1. Throughput de SHA-256 vs. tamaño de fragmento C para GPU v1/v2 (kernel-only y E2E, staging pinned) y serie de referencia CPU SHA-NI 4T evaluada a cada tamaño C, incorporando barras de error con la desviación estándar.

Como sintetizan la Figura 1 y la Tabla II, la GPU v2 supera a la CPU SHA-NI 4T en throughput E2E únicamente en el escenario de fragmentos mínimos $C = 64$ B (4.98 GB/s frente a 3.55 GB/s). Para todo $C \geq 1$ KB, la CPU SHA-NI 4T supera con claridad el rendimiento E2E de la GPU (alcanzando 8.21 GB/s en 1 KB y 8.99 GB/s en 1 MB), beneficiándose de operar en la memoria principal del host sin cruzar el enlace PCIe.

IV-C. Comparación CPU vs. GPU

La Tabla IV consolida las configuraciones evaluadas para el escenario principal $C = 64$ B, destacando tres conclusiones principales:

- **Rendimiento E2E y transferencia de bus:** En mensajes mínimos ($C = 64$ B), la GPU v2 E2E con memoria pinned supera a la referencia CPU SHA-NI 4T. No obstante, al activar todos los núcleos disponibles de la CPU, tanto SHA-NI 8T como la configuración heterogénea 12 CPUs (P+E) bajo planificación dinámica revierten la ventaja y sobrepasan el throughput E2E de la GPU, al beneficiarse de acceso directo a la memoria principal del sistema sin sufrir la saturación del bus periférico PCIe.
- **Eficiencia energética de dispositivo vs. sistema completo:** Como dispositivo aislado, la GPU en modo E2E (3.49 J/GB) consume un 15.5 % menos de energía que la referencia CPU SHA-NI 4T (4.13 J/GB, razón de 0.85×), pero frente a la configuración CPU 12 CPUs (P+E) (2.53 J/GB) la GPU aislada consume un 37.9 % más (1.38×). Esto indica que, frente a la mejor CPU, la GPU E2E es menos eficiente incluso considerándola como dispositivo aislado. Al evaluar el sistema completo, el conjunto GPU+host alcanza 5.48 J/GB frente a los 4.13 J/GB de SHA-NI 4T (+32.7 %, 1.33×) y los 2.53 J/GB de 12 CPUs (P+E) (+116.6 %, 2.17×), consolidando a la CPU heterogénea como la arquitectura más eficiente tanto a nivel de dispositivo como de plataforma completa.
- **Potencial aritmético de cómputo puro:** En régimen kernel-only (datos residentes en VRAM), la GPU v2 exhibe una tasa de procesamiento muy superior a cualquiera de las configuraciones de CPU. Esta ventaja sugiere como hipótesis que la GPU resultaría intrínsecamente más eficiente energéticamente si las transferencias periféricas fuesen amortizadas por algoritmos criptográficos iterativos de múltiples rondas locales, mientras que para flujos de red o transacciones independientes de un solo paso, el bus PCIe determina el rendimiento final.

Tabla IV
COMPARACIÓN CRUZADA DE RENDIMIENTO Y EFICIENCIA ENERGÉTICA ($C = 64$ B)

Plataforma / Config.	Throughput (GB/s)	Mhash/s	J/GB neto (disp.)	J/GB neto (+host)	Speedup[a]
Escalar 1T	0.11 ± 0.00	1.71 ± 0.00	45.58 ± 0.05	45.58 ± 0.05	0.03×
SHA-NI 4T (ref.)	3.55 ± 0.00	55.45 ± 0.01	4.13 ± 0.05	4.13 ± 0.05	1.00×
SHA-NI 8T	5.60 ± 0.02	87.45 ± 0.35	2.67 ± 0.05	2.67 ± 0.05	1.58×
SHA-NI 12 CPUs (P+E)	5.84 ± 0.02	91.25 ± 0.24	2.53 ± 0.02	2.53 ± 0.02	1.65×
GPU v2 kernel	15.62 ± 0.15	244.02 ± 2.38	1.62 ± 0.02	2.45 ± 0.04	4.40×
GPU v2 E2E pinned	4.98 ± 0.02	77.86 ± 0.33	3.49 ± 0.04	5.48 ± 0.07	1.40×
FPGA (Spartan-7)	TODO:	TODO:	TODO:	TODO:	TODO:

[a] Speedup vs. CPU SHA-NI 4T E2E ($C = 64$ B). El speedup de GPU v2 kernel (aislado en VRAM) no es comparable con E2E. J/GB neto (+host) suma la energía RAPL PKG del host durante la ejecución (excluye la DRAM del host).

IV-D. Resultados de FPGA

La Tabla V presentará los resultados post-implementación para el acelerador SHA-256 RTL personalizado sintetizado en la FPGA AMD/Xilinx Spartan-7.

Tabla V
RESULTADOS DEL ACELERADOR RTL SHA-256 EN FPGA (XILINX SPARTAN-7)

Parámetro	Valor
Dispositivo Objetivo	Xilinx Spartan-7 (xc7s50)
HDL	Verilog RTL
$f_{\text{máx}}$ (MHz)	TBD
Ciclos de Reloj / Bloque	TBD
Throughput (Mbps)	TBD
Utilización de LUT	TBD / 32,600 (TBD %)
Utilización de FF	TBD / 65,200 (TBD %)
DSPs Usados	TBD
Potencia en el chip (W)	TBD
Eficiencia Energética (J/GB)	TBD
Speedup vs. CPU SHA-NI 4T	TBD

La implementación FPGA se enfoca en maximizar la eficiencia energética mediante control RTL determinista: la ruta de datos dedicada elimina la sobrecarga de recuperación de

instrucciones y el costo del predictor de saltos inherente a los procesadores de propósito general, a la vez que evita el cuello de botella de la transferencia PCIe que limita el throughput efectivo de la GPU.

V. Conclusiones

Este artículo evaluó empíricamente el algoritmo criptográfico SHA-256 bajo un marco heterogéneo comparando CPU con extensiones SHA-NI, GPU bajo modelo SIMT y FPGA personalizada en RTL, sobre un flujo masivo de mensajes independientes de 64 B. Las mediciones en la plataforma evaluada indican que, si bien la GPU ofrece un potencial aritmético masivo en cómputo puro en VRAM (15.62 GB/s), su tasa extremo a extremo queda estrangulada por el bus PCIe a 4.98 GB/s. En contrapartida, la CPU heterogénea asistida por planificación dinámica libre de cerrojos y operando sobre memoria local supera el throughput E2E de la GPU con 5.84 GB/s y consolida la mayor eficiencia energética global de sistema completo (2.53 J/GB netos con host frente a 5.48 J/GB de GPU+host). Como trabajo futuro, la caracterización final del acelerador RTL en FPGA permanece en curso (**TODO: integrar resultados finales post-implementación de throughput, potencia en chip y recursos LUT/FF/DSP en Spartan-7**), cuya incorporación consolidará la comparativa sistemática de tres vías.

Referencias

[1] National Institute of Standards and Technology (NIST), "FIPS PUB 180-4: Secure hash standard (SHS)," U.S. Department of Commerce, Tech. Rep., 2015.

[2] ——, "Transitioning the use of cryptographic algorithms and key lengths," U.S. Department of Commerce, NIST Special Publication 800-131A, Revision 2, 2019.

[3] A. M. Antonopoulos, *Mastering Bitcoin: Programming the Open Blockchain*, 2nd ed. Sebastopol, CA, USA: O'Reilly Media, 2017.

[4] G. Wood, "Ethereum: A secure decentralised generalised transaction ledger," Ethereum Project Yellow Paper, 2014.

[5] M. Laimon, R. Almadadha, and S. Goh, "Energy consumption of crypto mining: Consequences and sustainable solutions using systems thinking and system dynamics analysis," *Sustainability*, vol. 17, p. 3522, 2025.

[6] R. Martino and A. Cilardo, "SHA-2 acceleration meeting the needs of emerging applications: A comparative survey," *IEEE Access*, vol. 8, pp. 28 415–28 436, 2020.

[7] H. E. Michail, A. Kakarountas, A. Milidonis, and C. Goutis, "A top-down design methodology for ultrahigh-performance hashing cores," *IEEE Trans. Dependable Secure Comput.*, vol. 6, no. 4, pp. 255–268, 2009.

[8] R. Chaves, G. Kuzmanov, L. Sousa, and S. Vassiliadis, "Improving SHA-2 hardware implementations," in *CHES 2006 - Workshop on Cryptographic Hardware and Embedded Systems*. Springer, 2006, pp. 298–310.

[9] ——, "Cost-efficient SHA hardware accelerators," *IEEE Trans. Very Large Scale Integr. (VLSI) Syst.*, vol. 16, no. 8, pp. 999–1008, 2008.

[10] T. Li, C. Cheng, R. Wang, C. Wang, X. Zou, and D. Yu, "A high performance hardware implementation of SHA-256 algorithm," in *2024 IEEE 4th Int. Conf. Information Technology, Big Data and Artificial Intelligence (ICIBA)*, 2024, pp. 477–481.

[11] P. Nannipieri, M. Bertolucci, L. Baldanzi, L. Crocetti, S. D. Matteo, F. Falaschi, L. Fanucci, and S. Saponara, "SHA2 and SHA-3 accelerator design in a 7 nm technology within the European Processor Initiative," *Microprocessors and Microsystems*, vol. 87, p. 103444, 2021.

[12] R. Martino and A. Cilardo, "Designing a SHA-256 processor for blockchain-based IoT applications," *Internet of Things*, vol. 11, p. 100254, 2020.

[13] B. Kieu-Do-Nguyen, T.-T. Hoang, C.-K. Pham, and C. Pham-Quoc, "A power-efficient implementation of SHA-256 hash function for embedded applications," in *2021 Int. Conf. Advanced Technologies for Communications (ATC)*, 2021, pp. 39–44.

[14] M. A. Hitesh, R. Mahendra, H. S. Sumanth, and S. K. Panda, "Secure hash algorithm SHA-256 based hardware acceleration using Spartan-6," *Int. J. Microsystems and IoT*, vol. 1, no. 3, pp. 121–126, 2023.

[15] H. L. Pham, T. H. Tran, V. T. D. Le, and Y. Nakashima, "A high-efficiency FPGA-based multimode SHA-2 accelerator," *IEEE Access*, vol. 10, pp. 11 830–11 845, 2022.

[16] M. Kammoun, M. Elleuchi, M. Abid, and M. S. BenSaleh, "FPGA-based implementation of the SHA-256 hash algorithm," in *2020 IEEE Int. Conf. Design & Test of Integrated Micro & Nano-Systems (DTS)*, 2020, pp. 1–6.

[17] S. Mehammed and D. Lemma, "Improving the performance of proof of work-based Bitcoin mining using CUDA," *Int. J. Innov. Sci. Res. Technol. (IJISRT)*, vol. 6, no. 12, pp. 1138–1143, 2021.

[18] S. Mehammed, "Optimizing proof-of-work for secure health data blockchain using compute unified device architecture," *Blockchain in Healthcare Today*, 2025, PMCID: PMC12860445.

[19] S. Gueron and V. Krasnov, "Simultaneous hashing of multiple messages," Cryptology ePrint Archive, Report 2012/371, 2012.

[20] S. Gulley, V. Gopal, K. Yap, W. Feghali, J. Guilford, and G. Wolrich, "Intel SHA extensions: New instructions supporting the secure hash algorithm on Intel architecture processors," Intel Corporation, White Paper, 2013.

[21] Z. Zhang and P. Liu, "A hybrid-CPU-FPGA-based solution to the recovery of sha256crypt-hashed passwords," *IACR Trans. Cryptogr. Hardw. Embed. Syst.*, vol. 2020, no. 4, pp. 1–23, 2020.

[22] H. L. Pham, T. H. Tran, T. D. Phan, V. T. D. Le, D. K. Lam, and Y. Nakashima, "Double SHA-256 hardware architecture with compact message expander for Bitcoin mining," *IEEE Access*, vol. 8, pp. 140 410–140 424, 2020.

[23] A. K. Pham, V. T. Vo, V. T. D. Le, T. H. Vu, H. L. Pham, V. T. Nguyen, and Y. Nakashima, "Crypto-RV: High-efficiency FPGA-based RISC-V cryptographic co-processor for IoT security," arXiv preprint arXiv:2602.04415, 2026.

[24] L. Gaspar, "FPGA performances in cryptography: Assessment of hash functions and symmetric cryptography on FPGA," Joint Research Centre, European Commission, JRC Technical Report JRC91528, 2014.